Machine Exercise 2

Carl Justin de Guia, 2011-06521

```
clear
clc
close all
```

Problem 1: Subspace Identification Methods (SIM)

We'll start by splitting the data into training and validation sets for both the small and large dataset.

```
load("p_dat.mat", "p_dat")

% 1001 samples, t = 0 to 500 s.
Ts = 0.5;
t = (0:1000)' * Ts;

% split - small vs large
small = p_dat{1};
large = p_dat{2};

% split train vs validation
small_train = small(1:501, :);
small_val   = small(502:end, :);

large_train = large(1:501, :);
large_val   = large(502:end, :);

t_train = t(1:501);
t_val   = t(502:end);

% split input vs output
u_small_train = small_train(:, 1:3);
y_small_train = small_train(:, 4:6);

u_large_train = large_train(:, 1:3);
y_large_train = large_train(:, 4:6);

u_small_val = small_val(:, 1:3);
y_small_val = small_val(:, 4:6);

u_large_val = large_val(:, 1:3);
y_large_val = large_val(:, 4:6);

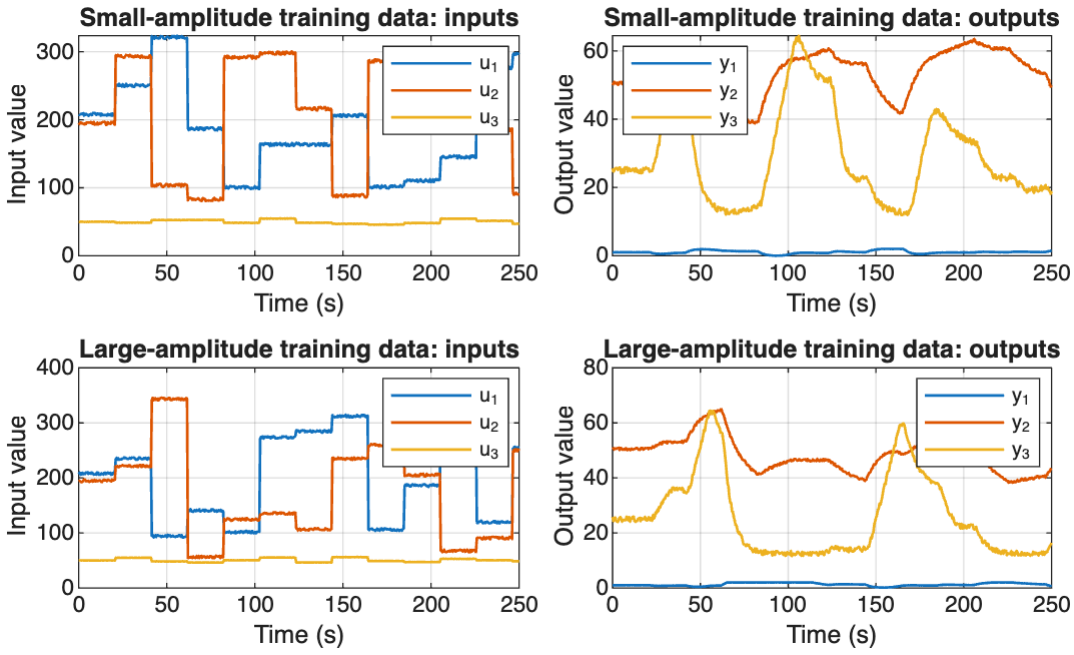

%% Inspect the 3-input, 3-output training data
figure
tiledlayout(2, 2, "TileSpacing", "compact", "Padding", "compact")
```

```
% Small-amplitude inputs
nexttile
plot(t_train, u_small_train, "LineWidth", 1)
grid on
title("Small-amplitude training data: inputs")
xlabel("Time (s)")
ylabel("Input value")
legend("u_1", "u_2", "u_3", "Location", "best")

% Small-amplitude outputs
nexttile
plot(t_train, y_small_train, "LineWidth", 1)
grid on
title("Small-amplitude training data: outputs")
xlabel("Time (s)")
ylabel("Output value")
legend("y_1", "y_2", "y_3", "Location", "best")

% Large-amplitude inputs
nexttile
plot(t_train, u_large_train, "LineWidth", 1)
grid on
title("Large-amplitude training data: inputs")
xlabel("Time (s)")
ylabel("Input value")
legend("u_1", "u_2", "u_3", "Location", "best")

% Large-amplitude outputs
nexttile
plot(t_train, y_large_train, "LineWidth", 1)
grid on
title("Large-amplitude training data: outputs")
xlabel("Time (s)")
ylabel("Output value")
legend("y_1", "y_2", "y_3", "Location", "best")
```



Now that we've split the dataset by input amplitude, training vs validation and input vs output, we can start estimating the params using the System Identification Toolbox's Canonical Variate Analysis.

To start, we estimate CVA models of orders 1-10 using the training data. For each order, we calculate the fit for all three outputs on both the training and validation datasets, then take the mean of the three fit values. The order with the highest mean validation fit is selected as the best order because it performs best on unseen data. All of this is done in the `find_best_order` function.

```
test_orders = 10;

function results = find_best_order(z_train, z_val, test_orders, method)
    results.models = cell(test_orders, 1);
    results.fit_train = zeros(test_orders, 3);
    results.fit_val = zeros(test_orders, 3);

    options = n4sidOptions("N4Weight", method, "Display", "off");

    for order = 1:test_orders
        sys = n4sid(z_train, order, options);

        [~, train_fit] = compare(z_train, sys);
        [~, val_fit] = compare(z_val, sys);

        results.models{order} = sys;
        results.fit_train(order, :) = train_fit(:).';
        results.fit_val(order, :) = val_fit(:).';
    end

    [results.best_fit_train, results.best_order_train] =
max(results.fit_train, [], 1);
```

```
    [results.best_fit_val, results.best_order_val] = max(results.fit_val,
[], 1);

    results.mean_fit_train = mean(results.fit_train, 2);
    results.mean_fit_val = mean(results.fit_val, 2);

    [results.best_mean_fit_train, results.best_mean_order_train] =
max(results.mean_fit_train);
    [results.best_mean_fit_val, results.best_mean_order_val] =
max(results.mean_fit_val);
end

function print_results(label, results, split)
    if split == "training"
        best_fit = results.best_fit_train;
        best_order = results.best_order_train;
        best_mean_fit = results.best_mean_fit_train;
        best_mean_order = results.best_mean_order_train;
    else
        best_fit = results.best_fit_val;
        best_order = results.best_order_val;
        best_mean_fit = results.best_mean_fit_val;
        best_mean_order = results.best_mean_order_val;
    end

    fprintf("\n%s\n", label)

    for output = 1:3
        fprintf("y_%d: best order = %d, fit = %.2f%%\n", output,
best_order(output), best_fit(output))
    end

    fprintf("Overall: best mean fit = %.2f%%, best order = %d\n",
best_mean_fit, best_mean_order)
end

z_small_train = iddata(y_small_train, u_small_train, Ts);
z_small_val = iddata(y_small_val, u_small_val, Ts);
z_large_train = iddata(y_large_train, u_large_train, Ts);
z_large_val = iddata(y_large_val, u_large_val, Ts);

small_cva = find_best_order(z_small_train, z_small_val, test_orders, "CVA");
large_cva = find_best_order(z_large_train, z_large_val, test_orders, "CVA");

print_results("Small input – training", small_cva, "training")
```

```
Small input – training
y_1: best order = 6, fit = 55.24%
y_2: best order = 10, fit = 85.90%
y_3: best order = 7, fit = 70.35%
Overall: best mean fit = 70.20%, best order = 6
```

```
print_results("Small input – validation", small_cva, "validation")
```

```
Small input – validation
y_1: best order = 4, fit = 49.00%
y_2: best order = 5, fit = 70.53%
y_3: best order = 5, fit = 56.43%
Overall: best mean fit = 57.60%, best order = 4
```

```
print_results("Large input – training", large_cva, "training")
```

```
Large input – training
y_1: best order = 8, fit = 57.40%
y_2: best order = 10, fit = 85.26%
y_3: best order = 7, fit = 61.58%
Overall: best mean fit = 67.26%, best order = 8
```

```
print_results("Large input – validation", large_cva, "validation")
```

```
Large input – validation
y_1: best order = 4, fit = 52.90%
y_2: best order = 4, fit = 72.51%
y_3: best order = 4, fit = 56.57%
Overall: best mean fit = 60.66%, best order = 4
```

Analyzing the outputs:

- For the small inputs, training set gets the best mean fit of 70.2% at order 6 while validation set gets 57.6% at order 4.
- For the large inputs, we get a 67.26% best mean fit on order 8 for the training set and 60.66% on order 4.

For better visualization, here's the plots of the output and model prediction on the training data and the validation data

```
small_sys = small_cva.models{small_cva.best_mean_order_val};
large_sys = large_cva.models{large_cva.best_mean_order_val};

fprintf("\nCVA model output vs measured output – Small input training
data\n")
```

```
CVA model output vs measured output – Small input training data
```

```
figure("Name", "CVA | Small input | Training", "NumberTitle", "off")
compare(z_small_train, small_sys)

fprintf("\nCVA model output vs measured output – Small input validation
data\n")
```

```
CVA model output vs measured output – Small input validation data
```

```
figure("Name", "CVA | Small input | Validation", "NumberTitle", "off")
compare(z_small_val, small_sys)

fprintf("\nCVA model output vs measured output – Large input training
data\n")
```

CVA model output vs measured output – Large input training data

```
figure("Name", "CVA | Large input | Training", "NumberTitle", "off")
compare(z_large_train, large_sys)

fprintf("\nCVA model output vs measured output – Large input validation
data\n")
```

CVA model output vs measured output – Large input validation data

```
figure("Name", "CVA | Large input | Validation", "NumberTitle", "off")
compare(z_large_val, large_sys)
```

Now let's compare the results of the CVA model with other model estimation algorithms, MOESP (Multivariable Output Error State Space) and SSARX (State-Space ARX model)

```
% 2.3.C: MOESP
small_moesp = find_best_order(z_small_train, z_small_val, test_orders,
"MOESP");
large_moesp = find_best_order(z_large_train, z_large_val, test_orders,
"MOESP");

% 2.3.D: SSARX
small_ssarx = find_best_order(z_small_train, z_small_val, test_orders,
"SSARX");
large_ssarx = find_best_order(z_large_train, z_large_val, test_orders,
"SSARX");


% Comparison
format_fit = @(fit, order) string(sprintf("%.2f%% (%d)", fit, order));

small_cva_fit = [format_fit(small_cva.best_mean_fit_train,
small_cva.best_mean_order_train); format_fit(small_cva.best_mean_fit_val,
small_cva.best_mean_order_val)];
small_moesp_fit = [format_fit(small_moesp.best_mean_fit_train,
small_moesp.best_mean_order_train);
format_fit(small_moesp.best_mean_fit_val, small_moesp.best_mean_order_val)];
small_ssarx_fit = [format_fit(small_ssarx.best_mean_fit_train,
small_ssarx.best_mean_order_train);
format_fit(small_ssarx.best_mean_fit_val, small_ssarx.best_mean_order_val)];

small_input_fit_comparison = table(small_cva_fit, small_moesp_fit,
small_ssarx_fit);
small_input_fit_comparison.Properties.VariableNames = {'CVA', 'MOESP',
'SSARX'};
small_input_fit_comparison.Properties.RowNames = {'Training', 'Validation'};
```

```
large_cva_fit = [format_fit(large_cva.best_mean_fit_train,
large_cva.best_mean_order_train); format_fit(large_cva.best_mean_fit_val,
large_cva.best_mean_order_val)];
large_moesp_fit = [format_fit(large_moesp.best_mean_fit_train,
large_moesp.best_mean_order_train);
format_fit(large_moesp.best_mean_fit_val, large_moesp.best_mean_order_val)];
large_ssarx_fit = [format_fit(large_ssarx.best_mean_fit_train,
large_ssarx.best_mean_order_train);
format_fit(large_ssarx.best_mean_fit_val, large_ssarx.best_mean_order_val)];

large_input_fit_comparison = table(large_cva_fit, large_moesp_fit,
large_ssarx_fit);
large_input_fit_comparison.Properties.VariableNames = {'CVA', 'MOESP',
'SSARX'};
large_input_fit_comparison.Properties.RowNames = {'Training', 'Validation'};

fprintf("\nSmall-input fit comparison\n")
```

```
Small-input fit comparison
```

```
disp(small_input_fit_comparison)
```

	CVA	MOESP	SSARX
Training	"70.20% (6)"	"71.37% (10)"	"70.68% (10)"
Validation	"57.60% (4)"	"61.10% (3)"	"64.61% (3)"

```
fprintf("\nLarge-input fit comparison\n")
```

```
Large-input fit comparison
```

```
disp(large_input_fit_comparison)
```

	CVA	MOESP	SSARX
Training	"67.26% (8)"	"66.07% (7)"	"65.91% (9)"
Validation	"60.66% (4)"	"58.05% (4)"	"41.94% (5)"

The tables show the fit percentage on the system's best model for both the small and large input. And from here we can observe that SSARX performs best on the small-input amplitudes but poorly on the large-input amplitudes. For those, CVA or MOESP performed better.

Also for all three algorithms, the order that maximized the training mean fit was higher than the order that maximized the validation mean fit. This shows that higher-order models were trying to fit features specific to the training data, rather than generalize to fit unseen data. This shows the risk of overfitting when model order is selected solely from training performance.

Problem 2: Unknown State-Space Model

To start estimation of the model parameters, we'll prepare the provided training dataset (s5.mat) which has 6 inputs and 6 outputs.

```
clearvars
close all
clc

warning('off', 'all')

load("s5.mat")

u_train = s5.Input;
y_train = s5.Out;

N = size(u_train, 1);

fprintf("Training samples: %d\n", N);
```

```
Training samples: 2001
```

```
fprintf("Number of inputs: %d\n", size(u_train, 2));
```

```
Number of inputs: 6
```

```
fprintf("Number of outputs: %d\n", size(y_train, 2));
```

```
Number of outputs: 6
```

Then, we'll setup the `evaluate_metrics` function which will compute the metrics to be used for training and validation. Specifically, it will return each of these scores per output column:

- **`MSE:`** average squared prediction error. Lower is better.
- **`NMSE:`** MSE relative to how much the true output naturally varies. Lower is better; below 1 means better than simply predicting the output's average.
- **`R2:`** fraction of output variation explained by the model. Closer to 1 is better; 0 means no better than predicting the average; negative means worse.

```
function metrics = evaluate_metrics(y_actual, y_predicted)
    error = y_actual - y_predicted;

    metrics.MSE = mean(error.^2, 1);

    metrics.NMSE = metrics.MSE ./ var(y_actual, 0, 1);

    metrics.R2 = 1 - sum(error.^2, 1) ./ sum((y_actual - mean(y_actual,
1)).^2, 1);
end
```

But before using this to evaluate the model's results with the validation data, we must first determine which SIM at what order can produce the best estimated parameters from our training data. And for that, the `find_best_sim` function is created to:

- create the model using `n4sid` provided the training input and output `z_train`, the SIM (either CVA, `MOESP` or SSARX) or PEM (AR, ARX, ARMA, ARMAX, Transfer function, Box-Jenkins) `method` , and the `order.`

- a model will be created, where `order` will be changed from 1 to test_orders.
- Compute for the order's mean fit percentage and use that to determine the best order to use.

```
function results = find_best_sim(z_train, test_orders, method)
    results.models = cell(test_orders, 1);
    results.fit_train = cell(test_orders, 1);
    results.mean_fit_train = zeros(test_orders, 1);

    for order = 1:test_orders
        switch method
            case {"CVA", "MOESP", "SSARX"}
                options = n4sidOptions("N4Weight", method, "Display", "off");
                sys_model = n4sid(z_train, order, options);
            case "ARX"
                sys_model = arx(z_train, [order order 1]);
            case "ARMAX"
                sys_model = armax(z_train, [order order order 1]);
            case "OE"
                sys_model = oe(z_train, [order order 1]);
            case "BJ"
                sys_model = bj(z_train, [order order order order 1]);
        end

        [~, train_fit] = compare(z_train, sys_model);

        results.models{order} = sys_model;
        results.fit_train{order} = train_fit;
        results.mean_fit_train(order) = mean(cell2mat(train_fit(:)));
    end

    [results.best_mean_fit_train, results.best_mean_order_train] =
max(results.mean_fit_train);
end
```

With the function setup, we can find the best order for each SIM, and use that for comparison to select the model to be used.

```
Ts = 0.05;
test_orders = 15;

z_train = iddata(num2cell(y_train, 1), num2cell(u_train, 1), Ts);
```

```
cva = find_best_sim(z_train, test_orders, "CVA");
moesp = find_best_sim(z_train, test_orders, "MOESP");
ssarx = find_best_sim(z_train, test_orders, "SSARX");

methods = ["CVA"; "MOESP"; "SSARX"];

best_mean_fit_train = [
    cva.best_mean_fit_train
    moesp.best_mean_fit_train
    ssarx.best_mean_fit_train
    ];

best_mean_order_train = [
    cva.best_mean_order_train
    moesp.best_mean_order_train
    ssarx.best_mean_order_train
    ];

sim_results = table(methods, best_mean_fit_train, best_mean_order_train);
sim_results.Properties.VariableNames = ["Method", "MeanTrainingFit",
"BestOrder"];
disp(sim_results)
```

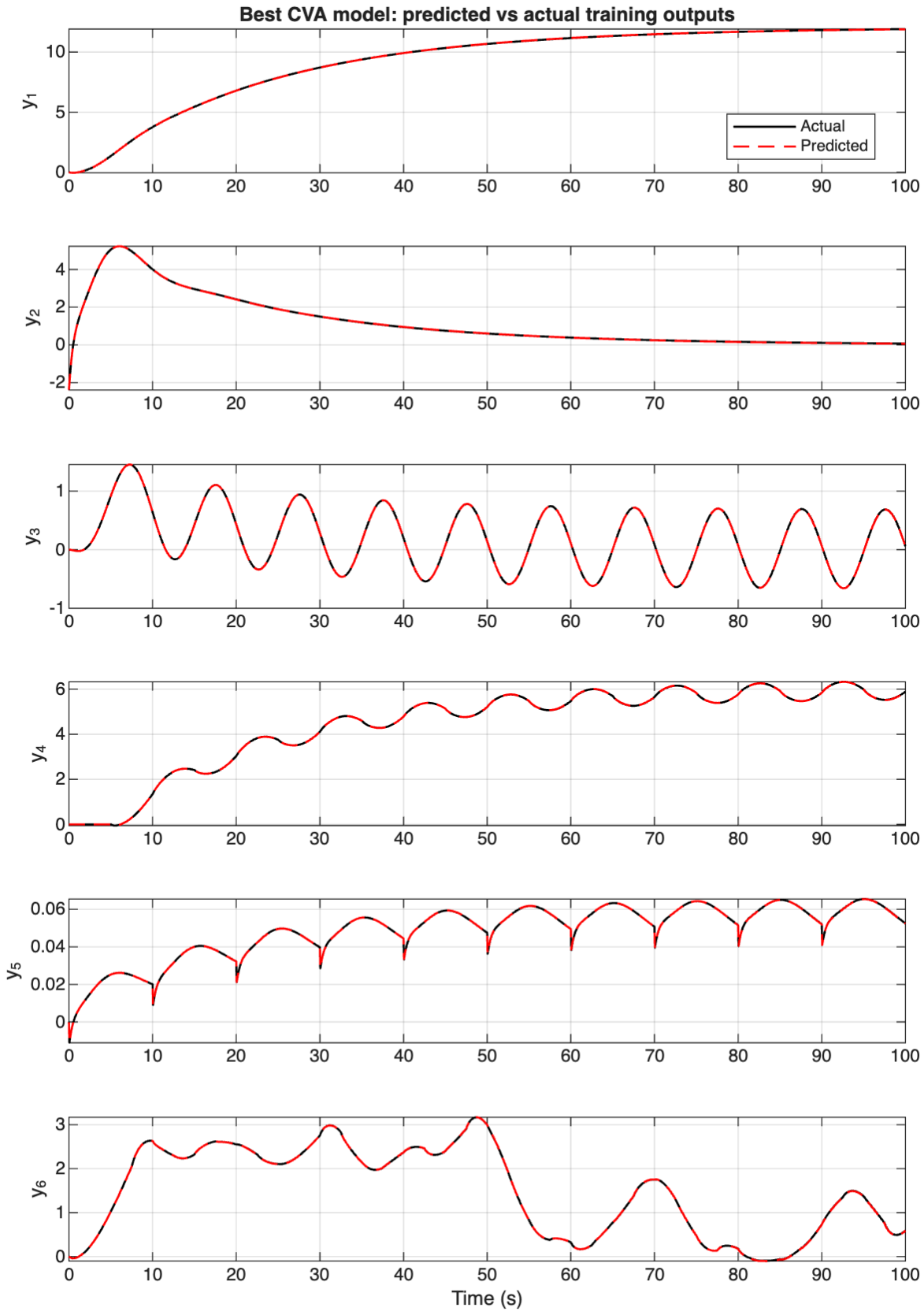
Method	MeanTrainingFit	BestOrder
"CVA"	99.68	9
"MOESP"	88.328	2
"SSARX"	99.268	6

From the results above, we can see that the CVA has the best training fit at 99.68% at order 9. Here's a visualization of the model's output:

```
best_cva_model = cva.models{cva.best_mean_order_train};
[y_cva_predicted, ~] = compare(z_train, best_cva_model);
time_train = (0:N-1)' * Ts;

figure("Position", [100, 50, 1000, 1400])
tiledlayout(6, 1)
for output = 1:size(y_train, 2)
    nexttile
    plot(time_train, y_train(:, output), "k", "LineWidth", 1)
    hold on
    plot(time_train, y_cva_predicted{output}.OutputData, "r--", "LineWidth",
1)
    grid on
    ylabel("y_" + string(output))
    if output == 1
        title("Best CVA model: predicted vs actual training outputs")
        legend("Actual", "Predicted", "Location", "best")
    end
end
```

```
xlabel("Time (s)")
```


Best CVA model: predicted vs actual training outputs
y1
y2
y3
y4
y5
y6
Actual
Predicted
Time (s)

Now, let's try out the Prediction Error Models (PEM) and find the best model to use:

```
arx_results = find_best_sim(z_train, test_orders, "ARX");
armax_results = find_best_sim(z_train, test_orders, "ARMAX");
oe_results = find_best_sim(z_train, test_orders, "OE");
bj_results = find_best_sim(z_train, test_orders, "BJ");

pem_methods = ["ARX"; "ARMAX"; "OE"; "BJ"];

pem_best_mean_fit_train = [
    arx_results.best_mean_fit_train
    armax_results.best_mean_fit_train
    oe_results.best_mean_fit_train
    bj_results.best_mean_fit_train
    ];

pem_best_mean_order_train = [
    arx_results.best_mean_order_train
    armax_results.best_mean_order_train
    oe_results.best_mean_order_train
    bj_results.best_mean_order_train
    ];

pem_results = table(pem_methods, pem_best_mean_fit_train,
pem_best_mean_order_train);
pem_results.Properties.VariableNames = ["Method", "MeanTrainingFit",
"BestOrder"];
disp(pem_results)
```

Method	MeanTrainingFit	BestOrder
"ARX"	99.675	15
"ARMAX"	99.515	15
"OE"	99.321	6
"BJ"	99.517	11

The four PEM methods achieved high fits on the training data so I'll be using them all for validation along with the CVA model. The table below shows the three validation metrics for these five models using their best orders from the training data:

```
validation_methods = ["CVA"; "ARX"; "ARMAX"; "OE"; "BJ"];

validation_models = {
    cva.models{cva.best_mean_order_train}
    arx_results.models{arx_results.best_mean_order_train}
    armax_results.models{armax_results.best_mean_order_train}
    oe_results.models{oe_results.best_mean_order_train}
    bj_results.models{bj_results.best_mean_order_train}
    };
```

```
load("s13.mat")
u_validation = s13.Input;
y_validation = s13.Out;
z_validation = iddata(num2cell(y_validation, 1), num2cell(u_validation, 1),
Ts);

validation_mse = zeros(numel(validation_models), 1);
validation_nmse = zeros(numel(validation_models), 1);
validation_r2 = zeros(numel(validation_models), 1);

for model = 1:numel(validation_models)
    [y_predicted, ~] = compare(z_validation, validation_models{model});
    y_predicted_matrix = zeros(size(y_validation));

    for output = 1:size(y_validation, 2)
        y_predicted_matrix(:, output) = y_predicted{output}.OutputData;
    end

    metrics = evaluate_metrics(y_validation, y_predicted_matrix);
    validation_mse(model) = mean(metrics.MSE);
    validation_nmse(model) = mean(metrics.NMSE);
    validation_r2(model) = mean(metrics.R2);
end

validation_results = table(validation_methods, validation_mse,
validation_nmse, validation_r2);
validation_results.Properties.VariableNames = ["Method", "MSE", "NMSE",
"R2"];
disp(validation_results)
```

Method	MSE	NMSE	R2
"CVA"	10.243	5808.7	-5810.6
"ARX"	11.276	6410.4	-6412.7
"ARMAX"	9.7536	5518	-5519.7
"OE"	17.534	10053	-10057
"BJ"	2.2772	1187.2	-1186.7

The results for the five models are not great. Ideally, MSE and NMSE should be close to zero and R_2 should be close to one. To investigate what the issue is, we'll try to visualize the predicted and expected outputs. I've selected CVA and Box-Jenkins models (the highest rated metric).

```
models_to_plot = {validation_models{1}; validation_models{5}};
plot_methods = ["CVA"; "BJ"];
time_validation = (0:size(y_validation, 1)-1)' * Ts;

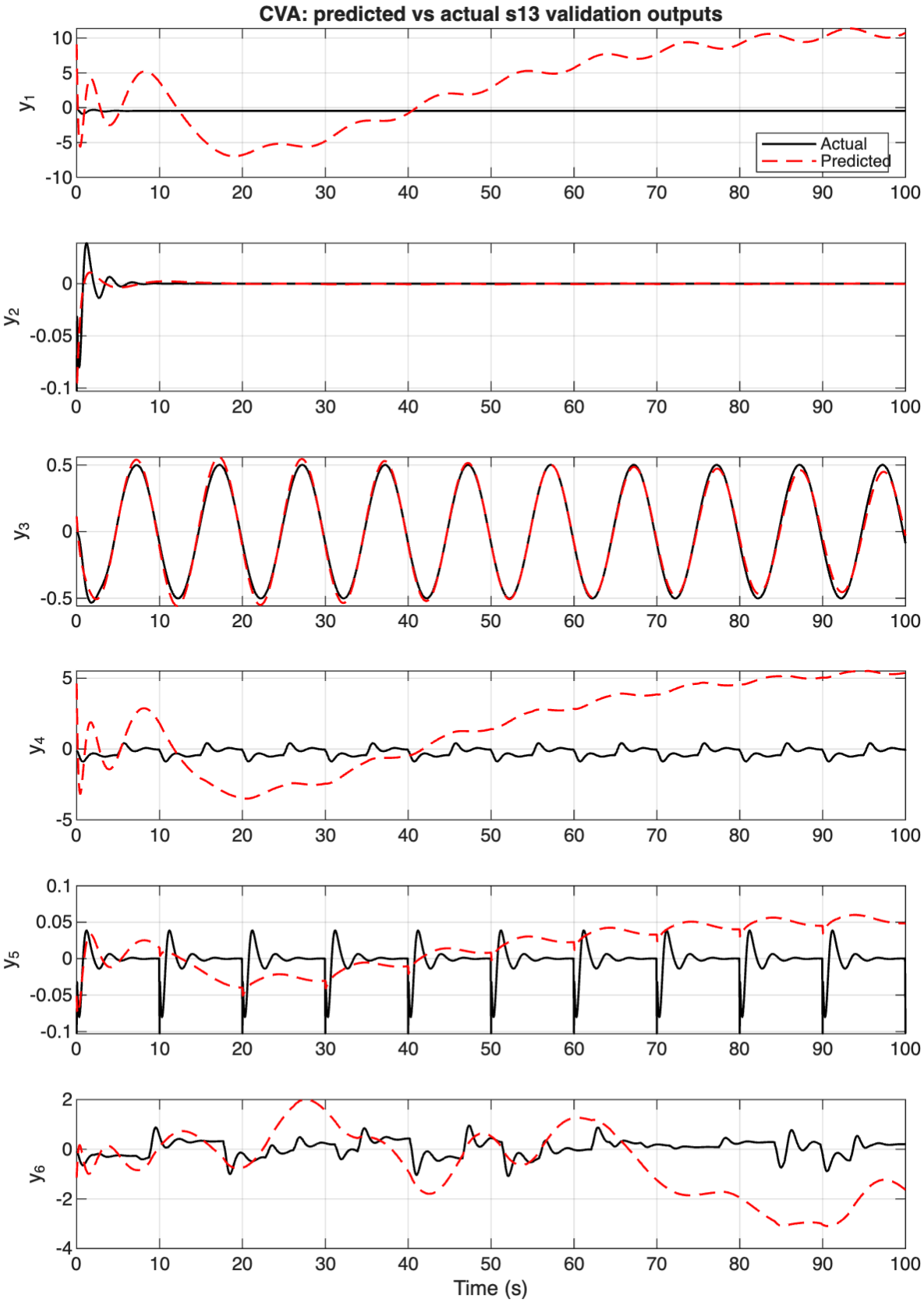
for model = 1:numel(models_to_plot)
    [y_predicted, ~] = compare(z_validation, models_to_plot{model});
```

```
    figure("Position", [100, 50, 1000, 1400])
    tiledlayout(6, 1)

    for output = 1:size(y_validation, 2)
        nexttile
        plot(time_validation, y_validation(:, output), "k", "LineWidth", 1)
        hold on
        plot(time_validation, y_predicted{output}.OutputData, "r--",
"LineWidth", 1)
        grid on
        ylabel("y_" + string(output))

        if output == 1
            title(plot_methods(model) + ": predicted vs actual s13
validation outputs")
            legend("Actual", "Predicted", "Location", "best")
        end
    end

    xlabel("Time (s)")
end
```


CVA: predicted vs actual s13 validation outputs
y1
y2
y3
y4
y5
y6
Actual
Predicted
Time (s)

We can see that both models were able to match y3 and roughly capture the initial transient of y2. For the other outputs, there are large errors which confirms the large R_2.

Although CVA and PEM models fitted the training set great, their performance on the blind validation set (high NMSE values, negative R_2 values) shows there are underlying factors (non-linear, time-varying, input-dependent) not able to be represented by classical linear models. For these type of cases, machine-learning models can better represent it.

And to test it out, we can use `nlarx`, a non-linear version of ARX. ARX predicts the current output using the past inputs and past outputs assuming their relationship is linear. NLARX still uses the same concept but learns nonlinear relations using a neural network. Below, we'll setup a two ReLU hidden-layer feed-forward NN that the NLARX will use to handle the non-linear mapping.

```
rng default

best_arx_model = arx_results.models{arx_results.best_mean_order_train};

neural_network = idNeuralNetwork([5 3], "relu",
NetworkType="RegressionNeuralNetwork");

neural_options = nlarxOptions(Focus="simulation", Display="off");

neural_nlarx = nlarx(z_train, [4 4 1], neural_network, neural_options);

[~, neural_train_fit] = compare(z_train, neural_nlarx);
neural_mean_training_fit = mean(cell2mat(neural_train_fit(:)));

neural_training_fit_by_experiment = table((1:numel(neural_train_fit))',
cell2mat(neural_train_fit(:)), VariableNames=["Experiment", "FitPercent"]);
disp(neural_training_fit_by_experiment)
```

Experiment	FitPercent
1	87.129
2	76.843
3	82.758
4	93.349
5	-539.06
6	72.175

```
comparison_methods = ["ARX"; "Neural NLARX"];

comparison_models = {
    arx_results.models{arx_results.best_mean_order_train}
    neural_nlarx
    };

comparison_training_fit = [
```

```
    arx_results.best_mean_fit_train
    neural_mean_training_fit
    ];

comparison_mse = zeros(numel(comparison_models), 1);
comparison_nmse = zeros(numel(comparison_models), 1);
comparison_r2 = zeros(numel(comparison_models), 1);

for model = 1:numel(comparison_models)
    [y_predicted, ~] = compare(z_validation, comparison_models{model});
    y_predicted_matrix = zeros(size(y_validation));

    for output = 1:size(y_validation, 2)
        y_predicted_matrix(:, output) = y_predicted{output}.OutputData;
    end

    metrics = evaluate_metrics(y_validation, y_predicted_matrix);

    comparison_mse(model) = mean(metrics.MSE);
    comparison_nmse(model) = mean(metrics.NMSE);
    comparison_r2(model) = mean(metrics.R2);

    if comparison_methods(model) == "Neural NLARX"
        y_neural_predicted = y_predicted;
    end
end

neural_comparison_results = table(comparison_methods,
comparison_training_fit, comparison_mse, comparison_nmse, comparison_r2);
neural_comparison_results.Properties.VariableNames = ["Method",
"MeanTrainingFit", "MSE", "NMSE", "R2"];

disp(neural_comparison_results)
```

Method	MeanTrainingFit	MSE	NMSE	R2
"ARX"	99.675	11.276	6410.4	-6412.7
"Neural NLARX"	-21.134	20.262	11924	-11929

```
time_validation = (0:size(y_validation, 1)-1)' * Ts;

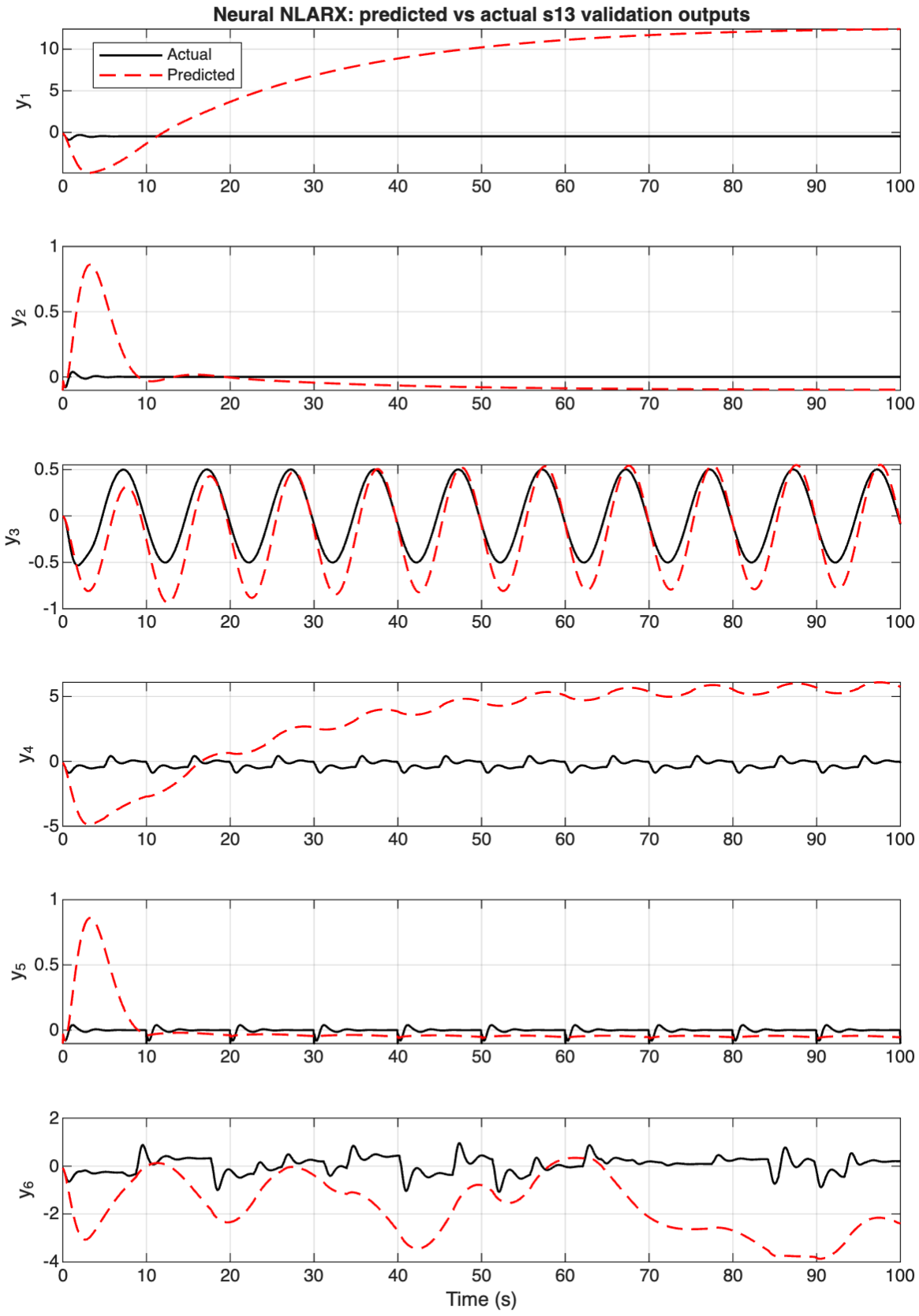
figure("Position", [100, 50, 1000, 1400])
tiledlayout(6, 1)

for output = 1:size(y_validation, 2)
    nexttile
    plot(time_validation, y_validation(:, output), "k", "LineWidth", 1)
    hold on
    plot(time_validation, y_neural_predicted{output}.OutputData, "r--",
"LineWidth", 1)
```

```
    grid on
    ylabel("y_" + string(output))

    if output == 1
        title("Neural NLARX: predicted vs actual s13 validation outputs")
        legend("Actual", "Predicted", "Location", "best")
    end
end

xlabel("Time (s)")
```


Neural NLARX: predicted vs actual s13 validation outputs
Actual
Predicted
y1
y2
y3
y4
y5
y6
Time (s)

Unfortunately, this solution did not outperform the classical methods. Its mean training fit was -21.13%, compared with 99.68% for ARX, and its blind-validation MSE was also higher. One analysis of the output `y3,` it shows unstable drift and incorrect amplitudes. Therefore, for the available data and selected structure, the neural NLARX model did not provide an improvement over the SIM and PEM models.

Reflection and Academic Integrity Declaration

Approach Summary

Basically we tested out different methods of modeling systems using Subspace Identification Models (SIMs) which tried to replicate model by predicts the current output using the past inputs and past outputs, and Prediction Error Models (PEMs) which chooses the model parameters by adjusting them until the predicted and actual attributes are as close as possible.

To do this:

- Setup the training and validation datasets. For the first problem, the dataset is split in half for its training and validation data. For the second problem, separate datasets for training and validation are used.
- Use training dataset to train the model. The toolkit functions `n4sid` (SIMs), `arx, armax, oe, bj` (PEMs) were used for this. Training is done using different orders, and the best order is determined based on the percentage fit (estimated output vs. the expected output).
- Once the model and the order is setup, the validation set is used to actually see how accurate the model is on untrained data, and the MSE, NMSE, and R2 metrics are computed to quantify how correct the models are.

For the first problem, I was able to properly train the models to acceptable results, but on the second one we failed to successfully simulate the model using both SIMs and PEMs. a different variant of the ARX which uses neural networks for handling non-linear relationships NLARX was also used to simulate the model but unfortunately that didn't over-perform compared to the classical options.

Key Insight

- Validation is always an integral part of training to ensure that models don't overfit and can be used even on untrained data.
- There are multiple factors why classical models like SIMs and PEMs can't simulate models, from non-linearity to input-dependence. But this doesn't also mean that introducing more advanced models or features like Machine Learning or Neural Networks can easily simulate the systems.

Most Difficult Part

I initially planned on implementing the Machine Exercise using Python, but the lack of SIMs and PEMs from trusted libraries became a huge time sink and blocker so I moved to use MATLAB instead. For the actual exercise, researching the machine learning solutions to use was difficult and the model I selected (NLARX) didn't return better results. On a more technical side, trying to reduce duplicate code especially with training and validating multiple models I took a bit of time trying to create functions that can be reused to reduce code bloat.

Disclosure on the use of AI Systems

- **Plot Enhancements** - Codex - I used this mostly for assisting in generating the charts and tables to visually show the results.
- **Research and Reference Lookup** - Codex - I used Codex for helping easily search all our notes as well as MATLAB documentation to speed up research time and focus more on implementation.